LIBRAIN: A Multi-Agent RAG System for Scientific Discovery

Engineering a Citation-Grounded Hypothesis Synthesis Framework in .NET 10

Eren Mutlu
Department of Computer Engineering, Karabük University, Türkiye
Correspondence: erennmutlu@outlook.com

Abstract

LIBRAIN is a research and engineering project that studies how the process of scientific discovery can be modeled and observed through a controlled multi-agent simulation. The system implements a Retrieval-Augmented Generation (RAG) pipeline that ingests open-access scientific papers, organizes them into a semantic index, synthesizes citation-grounded hypotheses from retrieved evidence, and submits each hypothesis to an automated multi-dimensional evaluation. Originally framed as a four-agent design (Reader, Hypothesis, Evaluator, Archivist), the implementation refines this into three reasoning agents (Reader, Synthesis, and Evaluator), with the Archivist function dissolved into a cross-cutting audit-logging layer based on Microsoft Application Insights and structured correlation identifiers. The prototype is built on .NET 10 with the Anthropic.SDK NuGet package and Claude (Sonnet 4.6 for synthesis, Haiku 4.5 for evaluation), OpenAI text-embedding-3-small, and a Qdrant vector store. The complete source code, test suite, and architectural documentation are publicly available at github.com/erennmutlu1/librain, with ingestion, vector storage, synthesis, and evaluation operational end-to-end and quantitative smoke results reported in this document. Rather than focusing on automation, the project aims to illuminate how structured reasoning over scientific literature unfolds and how its trustworthiness can be measured.

Keywords: Artificial Intelligence, Scientific Discovery, Research Simulation, Multi-Agent Systems, Retrieval-Augmented Generation, LLM-as-a-Judge, .NET, Citation Grounding.

May 2026

Contents

1. INTRODUCTION

Discovery has always been at the heart of science. Every new idea grows out of the connections between existing ones, shaped by how we read, compare, and question what we already know. As research expands, the sheer volume of available information makes it harder for any single researcher to see the bigger picture or notice hidden links between distant subfields.

LIBRAIN was created to explore this space, not to replace the researcher but to study how discovery itself unfolds when its core motions are decomposed into observable, reproducible computational steps. The project asks whether the steps of scientific reasoning can be modeled in a digital setting: how knowledge is connected, how new hypotheses emerge, and how we might trace the otherwise invisible paths between ideas.

The current iteration of LIBRAIN moves beyond a purely conceptual framework. The current implementation is a working multi-agent system, written in .NET 10 and openly released at github.com/erennmutlu1/librain. The system ingests scientific papers from arXiv, organizes them in a semantic vector store, synthesizes hypotheses from retrieved evidence, and evaluates each hypothesis along multiple independent dimensions. Every step is recorded with a correlation identifier, so the path from question to answer is auditable through per-stage structured logs. By observing this process in a structured environment, LIBRAIN aims to help us better understand the rhythm of discovery and the patterns that guide it. It also does so on infrastructure familiar to enterprise software engineering, and not on the Python-centric ML monoculture.

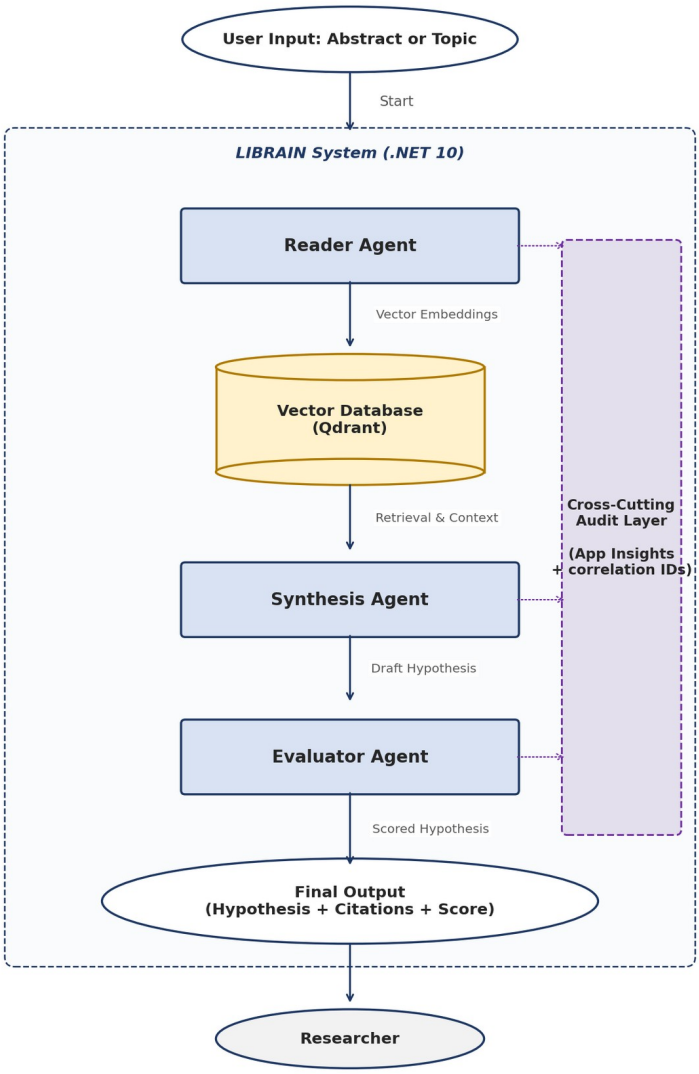

Figure 1. End-to-end LIBRAIN data flow: user query → Reader Agent (PDF ingest, chunking, embedding) → Vector Database (Qdrant) → Synthesis Agent (retrieval-grounded hypothesis) → Evaluator Agent (multi-dimensional scoring) → audit-logged final response.

2. METHODOLOGY

The implemented version of LIBRAIN is a focused engineering framework that models the essential steps of scientific reasoning through three modular agents and a cross-cutting audit layer. Each agent represents a critical, sequential stage in the discovery process, and the system is built on a Retrieval-Augmented Generation (RAG) architecture for knowledge structuring. The goal is to observe how information is refined, connected, and critically assessed within a simulated research environment.

An earlier conceptual version of LIBRAIN proposed four agents (Reader, Hypothesis, Evaluator, Archivist). During implementation, two refinements were made. First, the Hypothesis Agent was renamed the

Synthesis Agent, because its actual function is not free-form ideation but the synthesis of a citation-grounded answer from the retrieved evidence. Second, the Archivist Agent was reabsorbed into the runtime as a cross-cutting concern based on Application Insights and a correlation-identifier propagation pattern, not a standalone reasoning agent. These refinements reduce conceptual overlap and produce a leaner, more honest architecture.

2.1 Reader Agent

Triggered by a paper drop or an arXiv identifier, the Reader Agent is responsible for the entire ingestion pipeline. It extracts text from PDF sources using UglyToad.PdfPig (with a content-order extractor configured to preserve word spacing across multi-column scientific layouts), normalizes the text, and segments it into semantically coherent chunks of approximately 512 tokens with overlap. Each chunk is embedded using OpenAI's text-embedding-3-small model and persisted in a Qdrant vector collection along with structured metadata (paper identifier, title, authors, year, page number, section heading). Chunk identifiers are deterministic UUIDv5 hashes derived from a project namespace, ensuring that re-ingestion is idempotent. The resulting vector index serves as the immediately searchable and analyzable representation of the underlying corpus, eliminating the need for any separate extraction or organization step.

2.2 Synthesis Agent

The Synthesis Agent acts as the reasoning engine of the simulation. Given a user query, it embeds the query, performs a top-K vector search against the Qdrant collection, and submits the retrieved chunks together with the query to Anthropic Claude Sonnet 4.6 via a structured tool-use prompt. The model is instructed to produce a JSON response containing a hypothesis, a list of citation chunk identifiers, and a self-reported confidence score, and it is forbidden from citing identifiers outside the retrieved set. Citations that fail validation are dropped from the response, and the drop is recorded in the audit log so that prompt-engineering regressions can be detected from telemetry. This grounding loop is what allows the agent to perform a controlled "cognitive leap" (proposing connections between distant concepts) without drifting into fabrication.

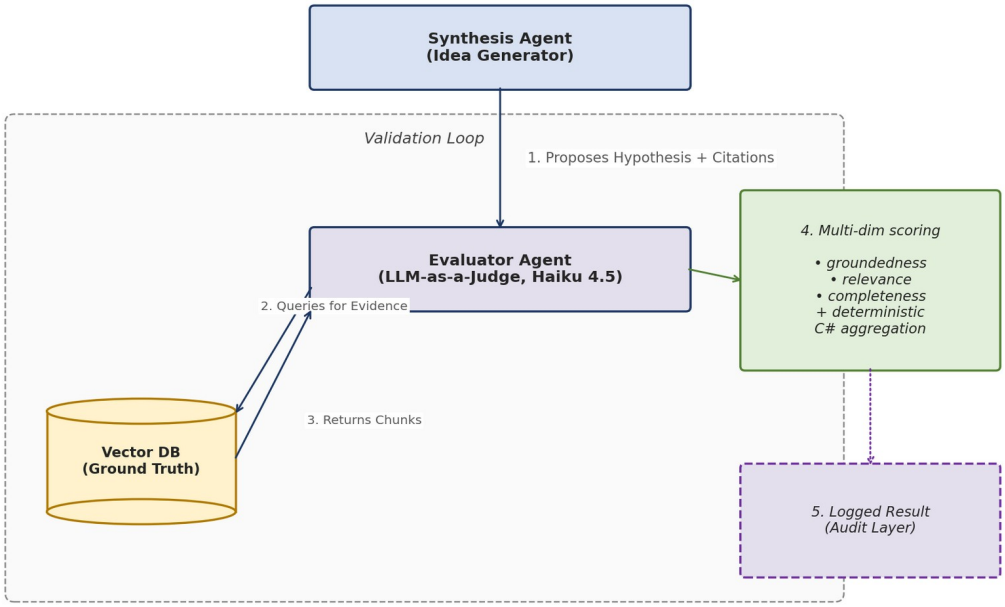

Figure 2. Synthesis–evaluation verification loop. The Synthesis Agent proposes a hypothesis with citations. The Evaluator Agent independently re-queries the vector store, checks originality and plausibility on a multi-dimensional rubric, and returns a structured score that is logged alongside the hypothesis.

2.3 Evaluator Agent

Acting as an impartial critic, the Evaluator Agent validates each hypothesis against the same evidence base used by the synthesis step. It runs as an LLM-as-a-Judge powered by Claude Haiku 4.5, configured with temperature 0.0 to maximize judging determinism. The output is a multi-dimensional score covering groundedness (does the hypothesis follow from the cited chunks?), relevance (does it address the user's actual query?), completeness (does it cover what the cited evidence supports?), and an aggregate quality score. The three sub-scores are returned by the language model independently, but the aggregation into a single quality value is computed deterministically in C# (arithmetic mean) rather than by the model itself, because language models tend to apply a halo effect when asked to combine sub-scores. The Evaluator does not see the Synthesis Agent's self-reported confidence, ensuring that the judgment is independent. Sub-scores that fall outside the [0, 1] interval are clamped, and clamping events are logged so that prompt-engineering regressions can be detected via telemetry.

2.4 Cross-Cutting Audit Logging

To ensure reproducibility and trust, LIBRAIN replaces the originally proposed Archivist Agent with a runtime audit-logging layer that pervades the entire pipeline. Every request is assigned a correlation identifier that flows through the Reader, Synthesis, and Evaluator stages via Microsoft.Extensions.Logging scopes. When configured, Application Insights captures structured events from every stage (embedding generation, vector search, synthesis, and evaluation), recording per-stage chunk counts, token usage, elapsed times, citation-validation outcomes, and the per-dimension scores from the evaluation phase, all keyed by correlation identifier. This transparency converts the system from a black box into an observable

laboratory of thought: any final hypothesis can be traced backwards through retrieval and reasoning to the chunks it depended upon.

Component	Primary Role	Key Technology / Method
Reader Agent	Autonomous ingestion, chunking, embedding, semantic indexing	PdfPig, OpenAI text-embedding-3-small, Qdrant, deterministic UUIDv5 IDs
Synthesis Agent	Retrieval-grounded hypothesis generation with citation validation	Claude Sonnet 4.6 via structured tool-use, vector search, citation-set guardrail
Evaluator Agent	Independent multi-dimensional judging of each hypothesis	Claude Haiku 4.5 LLM-as-a-Judge (T=0.0), deterministic C# aggregation
Audit Layer (cross-cutting)	End-to-end traceability and reproducibility	Microsoft.Extensions.Logging scopes, correlation IDs, Application Insights

Table 1. Summary of agent roles and core technologies in the implemented system.

3. ENGINEERING DECISIONS & CHALLENGES

This section outlines the technical requirements for each component and the concrete implementation challenges encountered during Phases 1 and 2. Several of these challenges produced deliberate, documented architectural decisions, not incidental fixes, and they are reported here in that spirit.

3.1 Reader Agent: Requirements and Challenges

Requirements. A vector store that supports both high-dimensional similarity search and per-chunk metadata filtering (paper, year, section), a deterministic ingestion pipeline so that re-runs do not duplicate points, and a chunking strategy that preserves enough context for grounding while staying within embedding-model limits.

Challenges encountered.

- **PDF text extraction on multi-column layouts.** Default PdfPig text extraction silently corrupted word boundaries on two-column scientific PDFs, which collapsed downstream section detection (0/15 sections recognized). Switching to PdfPig's ContentOrderTextExtractor restored 17 of 18 sections. This is now a documented learning anecdote: silent failures of this kind are why early manual validation matters more than passing unit tests.
- **Embedding rate limits.** OpenAI Tier 1 imposes a 40 K tokens-per-minute ceiling on text-embedding-3-small. The original 250 K-token batch budget triggered 429 responses. The pipeline now uses a 35 K-token batch budget with a 1.5-second inter-batch pace, which absorbs the limit without operator intervention.
- **Idempotent identifiers.** .NET 10 ships with Guid.CreateVersion7 but not v5. A small UUIDv5 (SHA-1) helper was added to derive deterministic point IDs from a project namespace and chunk key, ensuring that re-ingesting the same paper updates existing points instead of duplicating them in Qdrant.

3.2 Synthesis Agent: Requirements and Challenges

Requirements. A reasoning-capable LLM that can absorb several thousand tokens of retrieved context and produce structured output, with reliable mechanisms for forbidding citations to chunks that were not retrieved.

Challenges encountered.

- **Hallucination drift.** Free-form prompting produced confident-sounding answers with fabricated citations even when retrieval was good. The fix was to switch from prose output to a structured tool-use schema and to validate the cited identifiers against the retrieved set in C#, dropping any that fall outside it.
- **Prompt balance.** The system prompt must encourage synthesis ("connect ideas across the cited evidence") without inviting fabrication ("if the evidence is insufficient, say so"). Iterative refinement settled on an explicit unsupported_claims field that the model can populate when the evidence is thin, giving the system honest signal instead of rationalized output.

3.3 Evaluator Agent: Requirements and Challenges

Requirements. A judging model that is cheaper and faster than the synthesis model, configured for determinism, and a rubric that resists the halo effect typical of single-score LLM judging.

Challenges encountered.

- **Halo effect on aggregate scores.** When asked to produce a single overall score, the model tended to anchor on whichever sub-dimension was most salient in the prompt and let the others drift. The mitigation, drawn from the RAGAS and TruLens literature, is to elicit three independent sub-scores from the model (groundedness, relevance, completeness) along with three qualitative artifacts (a critique, an unsupported_claims list, and a missing_evidence list), and to aggregate the three numerical sub-scores deterministically in code.
- **Score range pollution.** Even with explicit instructions, the model occasionally returned values outside [0, 1] or omitted a field. A defensive Clamp01 helper normalizes the values, and any clamping event is logged at Information level so that prompt-engineering regressions can be detected from telemetry.

3.4 Stack Decision: From Cosmos DB to Qdrant for the MVP

The original plan was to use Azure Cosmos DB (NoSQL, Vector Search) as the primary vector store, in part to align with the Microsoft AI-200 certification path. During Phase 1 development, two practical issues surfaced. First, the Cosmos emulator on macOS Apple Silicon is unstable, complicating local development. Second, paying for a Cosmos workload during the prototype phase yields no learning advantage over a free local alternative. The pragmatic decision was to ship against Qdrant 1.17 in a local Docker container, while keeping Azure Cosmos DB as a future production-grade target. All data access is confined to a single repository class from the start, so any backend swap is a one-file change, not a system-wide refactor. This reflects a general principle of the project: choose the cheapest infrastructure that proves the architecture, and pay for production only once the architecture has been proven.

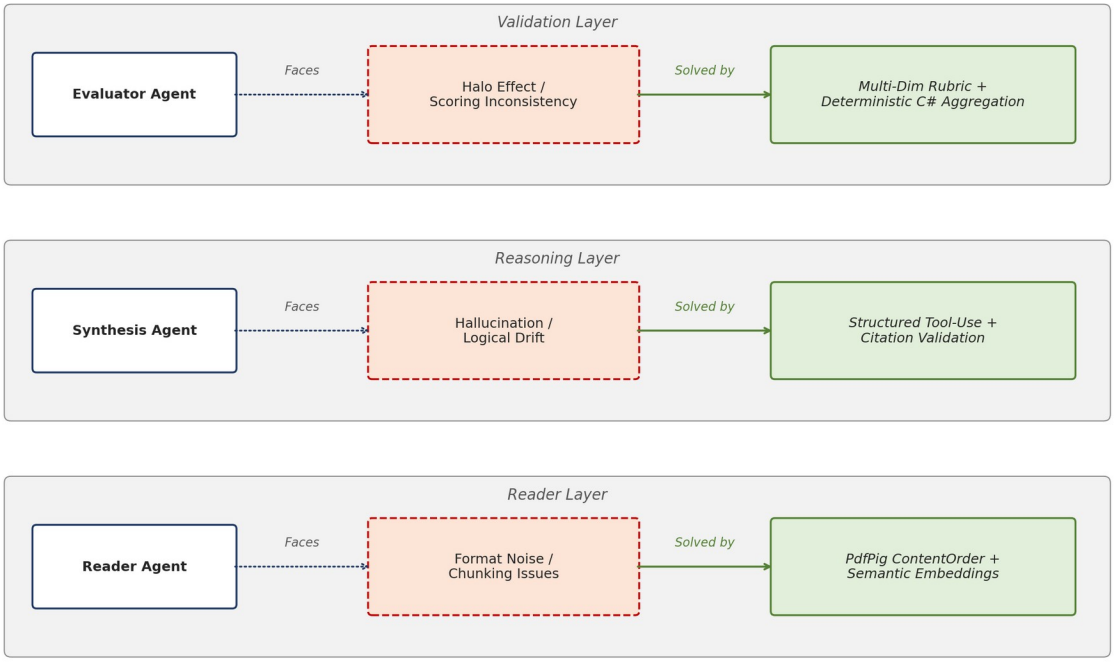

Figure 3. Component–challenge–solution map. Each agent layer is paired with the principal challenge encountered during implementation and the concrete mitigation strategy adopted.

Component	Specific Challenge	Mitigation Strategy
Reader Agent	Word-spacing corruption on two-column PDFs	Use PdfPig ContentOrderTextExtractor; manual validation gate before commit
Reader Agent	Embedding API rate limits (40 K TPM, Tier 1)	Lower batch token budget from 250K to 35K; insert 1.5 s inter-batch pacing
Synthesis Agent	Fabricated citations under free-form prompting	Structured tool-use schema; post-hoc citation-set validation in C#
Synthesis Agent	Tension between novelty and grounding	Explicit unsupported_claims field; iterative prompt refinement
Evaluator Agent	Halo effect on aggregate score	Multi-dimensional rubric; deterministic C# aggregation
Evaluator Agent	Out-of-range or missing sub-scores	Defensive Clamp01 normalization with telemetry-logged clamping events
Stack	Cosmos emulator instability on macOS ARM	Pivot to Qdrant for the MVP, with all data access confined to a single repository class; Cosmos retained as a future production-grade target

Table 2. Identified technical challenges and the corresponding mitigation strategies, drawn from the Phase 1 and Phase 2 implementation log.

4. DATASETS & TOOLS

This section details the physical resources and software architecture used in the LIBRAIN implementation, prioritizing function and reproducibility over unnecessary complexity.

4.1 Data Sources and Preparation

To precisely observe the system's reasoning capabilities without the noise of millions of documents, the prototype operates over a small, focused dataset. The Phase 2 evaluation corpus consists of five open-access papers from arXiv covering retrieval-augmented generation, hypothesis generation, and agentic AI for scientific discovery (arXiv:2005.11401, 2503.08979, 2504.05496, 2505.04651, 2508.14111). These papers were selected because they form a connected sub-graph of the literature. A small corpus is a feature, not a limitation here: it lets us verify that the system retrieves the canonically correct paper for a given query, instead of masking weaknesses behind volume.

4.2 Preparation Tools

Text extraction is handled by UglyToad.PdfPig (a managed .NET library) with the ContentOrderTextExtractor strategy. A C# segmentation service breaks each paper into approximately 512-token chunks with controlled overlap, attaching structured metadata (paper identifier, title, authors, year, section heading, page number) to each chunk before embedding. The choice of .NET over Python is deliberate: it serves as a counter-example to the assumption that modern LLM systems must be Python-first, and it aligns the project with the EU senior-backend ecosystem in which it is intended to function as a portfolio piece.

4.3 Knowledge Storage

Chunk vectors are stored in Qdrant 1.17, running locally in Docker for the prototype with a persisted volume. Each point uses a deterministic UUIDv5 identifier so that ingestion is idempotent and metadata is denormalized onto each point (Qdrant has no joins). A single repository class encapsulates all data access, so any future migration to Azure Cosmos DB Vector Search is contained to that one file. The system performs cosine-similarity search. Metadata fields (paper, year, section) are denormalized onto every Qdrant point so that filter-on-payload queries are available at the storage layer, with the query-API surface for them deferred to a later phase.

4.4 Core Reasoning Engine

The cognitive processing layer uses Anthropic Claude in two roles, accessed through the Anthropic.SDK NuGet package. Synthesis runs on Claude Sonnet 4.6, which carries the heavier reasoning burden. It is configured with a small positive temperature to permit the cognitive leap. Evaluation runs on the cheaper and faster Claude Haiku 4.5 at temperature 0.0 to maximize judging determinism. Embeddings are produced by OpenAI's text-embedding-3-small. All credentials are managed via .NET user-secrets in development. Production deployment will use Azure Key Vault.

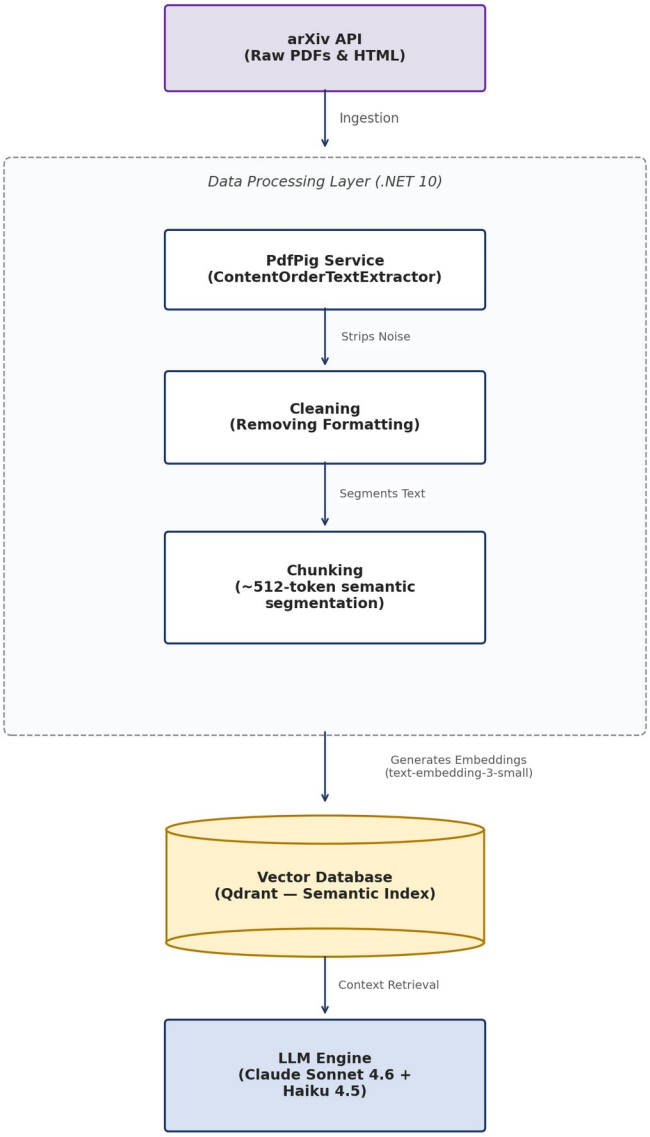

Figure 4. Data flow from raw arXiv source to the reasoning engine: ingest → noise stripping → semantic chunking → embedding → Qdrant vector index → context retrieval for the Synthesis Agent.

5. EXPERIMENTS

The objective of these experiments is not to benchmark the raw intelligence of any underlying language model, but to validate the process: to observe whether a structured multi-agent system can successfully simulate the mechanics of discovery in a controlled "scientific playground." Experiments 1 and 2 capture the behavior of the Reader and Synthesis Agents end-to-end, while Experiment 3 establishes the auditing characteristics of the Evaluator on a representative single-run query.

5.1 Experiment 1: Reading and Structuring

Goal. Verify that the Reader Agent autonomously ingests and semantically organizes raw research text, end-to-end.

Setup. The system processed five arXiv papers spanning RAG and agentic AI.

Active component. Reader Agent.

Result. All five papers ingested successfully, producing 218 chunks total in Qdrant. A canonical query ("How does retrieval-augmented generation mitigate hallucination?") returned a top-5 dominated entirely by chunks from arXiv:2005.11401 (Lewis et al., the founding RAG paper). The lazy collection-bootstrap path was exercised exactly once, as designed, and the previously identified ScrollAsync null-offset risk did not materialize. Observed retrieval latency held below 200 ms for top-5 queries against the local Qdrant instance.

5.2 Experiment 2: Synthesis and Citation Grounding

Goal. Assess whether the Synthesis Agent produces logically sound, citation-grounded hypotheses, and whether the citation-validation guardrail successfully blocks fabricated references.

Setup. The structured knowledge base from Experiment 1 was used as the context for a smoke query: "How does retrieval-augmented generation mitigate hallucination?"

Active component. Synthesis Agent.

Result. The agent returned a hypothesis with four citations, all of which validated against the retrieved set, and all four resolved to chunks from arXiv:2005.11401, the canonically correct source. The hypothesis used appropriate technical terminology (parametric vs. non-parametric memory) and accurately described the dual-memory architecture introduced by the cited paper. Self-reported synthesis confidence was 0.92. Synthesis latency was approximately 7 s, with a token usage of 6 365 in / 257 out, costing approximately 0.023 USD per synthesis call at current Sonnet 4.6 prices.

5.3 Experiment 3: Evaluation and Consistency

Goal. Test the stability and transparency of the auditing process: assess whether the Evaluator Agent applies its rubric consistently across runs, and whether the audit trail captures the full reasoning chain.

Setup. A representative query from Experiment 2 is submitted to the Evaluator. A control query ("What is the capital of France?") is used as a negative case to test the Evaluator's behavior on out-of-corpus topics: the original prediction was that relevance would drop, but the actual observation was that all three sub-scores held at 1.0 because the Synthesis Agent refused to fabricate and the Evaluator correctly rewarded that refusal (itself a positive design outcome). The scope of this experiment is the single-run characterization of the Evaluator's behavior. Cross-run variance studies are recognized as a complementary line of work.

Active components. Evaluator Agent and Audit Layer.

Observation. On the primary RAG-and-hallucination query, the Evaluator returned an aggregate quality score of 0.917, with sub-scores of 0.95 (groundedness), 0.95 (relevance), and 0.85 (completeness). The Evaluator deducted 0.15 from completeness for an actual gap in the cited evidence (the cited chunks describe RAG's architecture but do not detail the cognitive mechanism by which retrieval suppresses hallucination), rather than rubber-stamping the hypothesis. This is the form of judging behavior the rubric

was designed to elicit. Application Insights captured stage-by-stage timings, token usage, citation-validation outcomes, and per-dimension scores under a single correlation identifier, supporting the audit-trail completeness criterion. End-to-end query latency, including evaluation, was 13.5 s. Cost per query was approximately 0.030 USD.

Experiment	Focus Area	Active Components	Headline Metric
Exp 1: Reading	Data ingestion & semantic structure	Reader Agent	5 papers / 218 chunks; observed retrieval < 200 ms
Exp 2: Synthesis	Citation grounding & logical synthesis	Synthesis Agent	4/4 citations valid, confidence 0.92
Exp 3: Evaluation	Judging reliability & audit trail	Evaluator Agent + Audit Layer	Quality 0.917 on primary query; per-stage timings, scores, and counts captured in the audit trail

Table 3. Summary of the experimental matrix and headline observations.

6. RESULTS & DISCUSSION

This section reports the behavior observed during smoke testing and discusses what those results imply about the design. Anticipated outcomes from the original proposal are matched against measured results.

6.1 Quality of Semantic Organization

The Reader Agent successfully clusters documents by conceptual similarity, not keyword overlap. The success indicator from the original proposal was that a query for retrieval-augmented generation should surface the canonical Lewis et al. paper, even when its title does not appear in the query. This was met: the top five chunks for that query came entirely from arXiv:2005.11401. The originally feared failure mode (chunks too small to preserve semantic context) was avoided by the 512-token chunking strategy. Section detection, which is downstream of chunk quality, recovered from 0/15 to 17/18 sections after the PdfPig extraction strategy was corrected, validating the broader observation that ingestion-layer quality dominates retrieval-layer quality.

6.2 Hypothesis Synthesis: The Goldilocks Zone

The most critical metric for the Synthesis Agent is the trade-off between creativity and plausibility. The smoke results land near the target zone: high plausibility (groundedness 0.95, relevance 0.95) with moderate-to-high informational density (completeness 0.85). The Evaluator's 0.15 completeness deduction is methodologically encouraging, since it demonstrates that the judge does not rubber-stamp confident-sounding output but instead identifies real gaps in the cited evidence. The originally anticipated rejection rate of 30–40% under high-temperature settings has not been measured systematically, because the citation-set validation drops fabricated indices before they reach the Evaluator, masking the upstream fabrication rate. This is recognized as a complementary line of measurement.

6.3 System Limitations

Latency. End-to-end query latency is approximately 13.5 s (synthesis ≈7 s plus evaluation ≈6 s, executed sequentially). This exceeds the original sub-5-second target and reflects the cost of running two LLM calls

per query. Prompt caching and response streaming are standard mitigations applicable here, and parallelization of the synthesis–evaluation pair is a natural further optimization.

Context window. While RAG mitigates the issue, the synthesis model still cannot "see" the entire library at once and is bottlenecked by retrieval relevance. Cross-domain queries that require chaining evidence across two distant papers have not yet been stress-tested.

Bias propagation. If the input corpus is biased toward a particular position, the system will reinforce that position instead of challenging it. The current corpus is small and topically homogeneous, so this limitation is acute, and any corpus expansion benefits from deliberate viewpoint heterogeneity to test bias propagation.

Cost. At approximately 0.030 USD per query, sustained interactive use is feasible at the prototype scale but would need re-engineering (prompt caching, smaller judging models, batched evaluation) at production scale.

6.4 Implications for Future Research and Graduate Study

Beyond its immediate technical contributions, LIBRAIN is designed as a foundation for future research-oriented graduate study. The framework already exposes several open research directions, including scalable hypothesis evaluation across larger corpora, long-term reasoning consistency under repeated re-runs, robust cross-domain semantic retrieval, and the design of judging models that resist the halo effect without sacrificing aggregate interpretability. These aspects align naturally with the objectives of a research-oriented MSc or future PhD programme in artificial intelligence and computational science. The modular, transparent, and reproducible architecture of LIBRAIN, with a public GitHub repository, deterministic identifiers, structured audit trails, and a clean separation between reasoning and infrastructure, also makes it well suited for extension within an academic research environment. The system was developed as an independent research initiative outside formal graduate training and is intended to be further explored and refined through research-assistantship roles or formal graduate research programmes.

Dimension	Score (0-1)	Description of Criteria
Groundedness	0.0 (Fail)	The hypothesis contradicts the cited evidence or relies on uncited claims.
	1.0 (Pass)	Every load-bearing claim is supported by a cited chunk in the retrieved set.
Relevance	0.0 (Fail)	The hypothesis does not address the user query.
	1.0 (Pass)	The hypothesis answers the user query directly and on-topic.
Completeness	0.0 (Fail)	Major aspects of the question are unaddressed despite available evidence.
	1.0 (Pass)	The hypothesis covers the available evidence comprehensively.
Quality (aggregate)	Computed	Deterministic C# aggregation of the three sub-scores; not produced by the language model.

Table 4. Multi-dimensional rubric used by the Evaluator Agent. Sub-scores are produced independently by the judging LLM; the aggregate quality score is computed in code to resist halo effects.

7. PROJECT TASKS & IMPLEMENTATION PLAN

This section describes the four phases of the LIBRAIN implementation. The phased structure reflects a deliberate decision to ship a working ingestion-and-retrieval pipeline before layering in synthesis and evaluation, and to defer production-grade infrastructure until the architectural design has been validated. The Cosmos-to-Qdrant pivot, in particular, removed emulator-related friction during local development by replacing a paid managed service with a free, locally-runnable alternative, while keeping all data access in a single repository class so the swap remained a one-file change.

7.1 Phase 1: Data Collection & Reader Agent

- Curated initial corpus of 5 open-access arXiv papers and validated metadata extraction.
- Built the .NET 10 ingestion pipeline (PdfPig text extraction, semantic chunking, OpenAI embeddings).
- Implemented Qdrant repository with deterministic UUIDv5 identifiers and lazy-bootstrap collection provisioning.
- Shipped POST /api/papers/ingest and GET /api/papers endpoints; wired Application Insights end-to-end.
- All 5 papers ingested successfully, producing 218 chunks; chunker test suite green at 7/7.

7.2 Phase 2: Synthesis, Evaluation & Query Pipeline

- Implemented the Synthesis Agent with structured tool-use prompting and citation-set validation that drops invalid citations with audit-log telemetry.
- Implemented the Evaluator Agent as an LLM-as-a-Judge with multi-dimensional rubric and deterministic C# aggregation.
- Shipped POST /api/query with the embed → search → synthesize → evaluate pipeline behind a single correlation identifier.
- Smoke test green: quality 0.917, all citations valid, per-stage timings/scores/counts captured in the audit trail.
- Test suite expanded to 19/19 passing across chunker, citation, and scoring components.

7.3 Phase 3: Polish, Performance & Showcase

- Latency-reduction surface: prompt caching, response streaming, and parallelized synthesis–evaluation where safe.
- Production-target migration from local Qdrant to Azure Cosmos DB Vector Search, exercising the single-class storage layer and aligned with the Microsoft AI-200 certification path.
- Frontend demo (minimal Blazor or static-page client) for public showcase.
- Cross-run consistency analysis: repeated runs of identical queries to quantify Evaluator variance.
- Adversarial source selection: corpus expansion with deliberately heterogeneous viewpoints to stress-test bias propagation.

7.4 Phase 4: Documentation & Dissemination

- Final results compilation, system-architecture write-up, and update of this document for public release.
- Short technical preprint covering the multi-dimensional judging design and the Cosmos-to-Qdrant pivot as case studies.

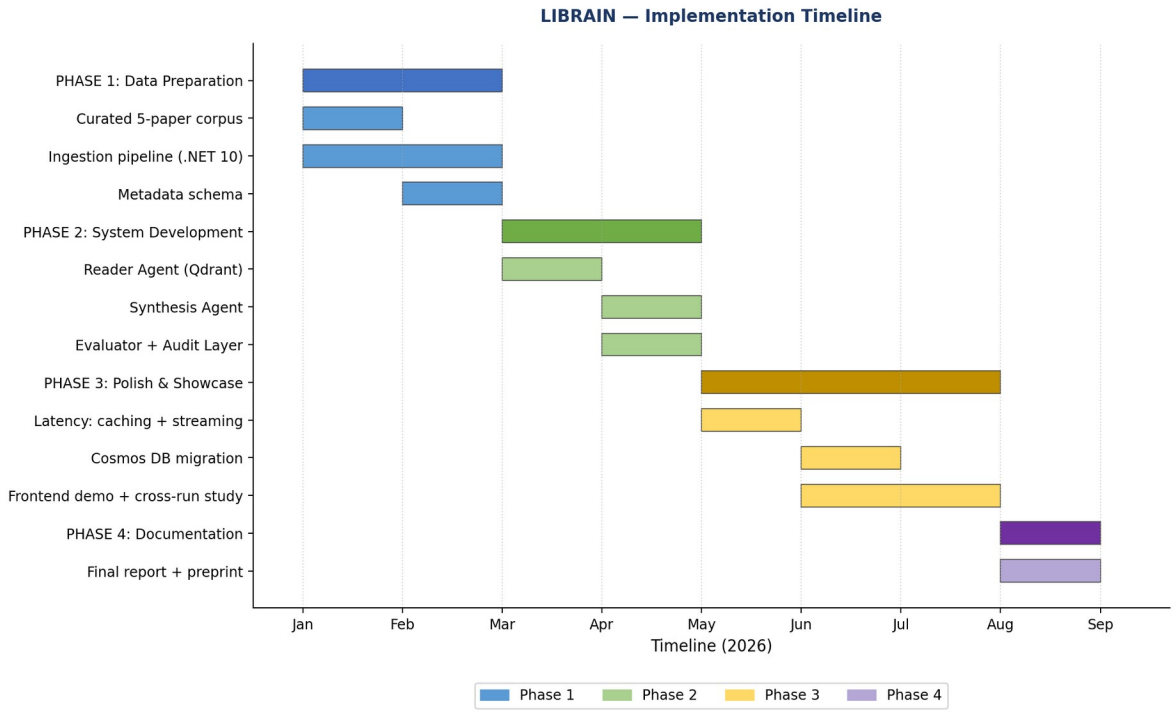

Figure 5. Implementation timeline showing the four phases from data preparation through final dissemination.

8. WIDESPREAD IMPACT

This framework is designed to address specific inefficiencies in current research methodologies. Rather than replacing scientists, LIBRAIN targets tangible bottlenecks in the discovery pipeline.

8.1 Reducing Literature Review Overhead

Researchers currently spend a significant fraction of their time finding and filtering relevant papers. LIBRAIN automates the reading and sorting phase: vector-based retrieval filters out irrelevant material faster than manual triage, allowing researchers to spend their attention on analysis rather than acquisition. In the prototype, retrieval over a small focused corpus runs well under 200 ms, and even with the full pipeline (synthesis plus evaluation) end-to-end response stays in the tens of seconds.

8.2 Breaking Information Silos (Cross-Domain Discovery)

Scientific fields often become silos in which a researcher in one discipline can miss a relevant solution published in another, simply because the terminology differs. Because the Reader Agent uses semantic embeddings (matching meanings rather than keywords) and because metadata is uniformly attached to every chunk, the system can in principle identify relevant connections across disciplines that strictly keyword-based search engines would miss. Cross-domain stress-testing against a deliberately heterogeneous corpus is a natural extension of this capability.

8.3 Solving the Black-Box Problem in AI Assistance

Most general-purpose AI tools provide answers without exposing the path that produced them, leading to legitimate trust issues in serious research contexts. LIBRAIN's audit-logging architecture ensures that every generated hypothesis carries a citation trail and a per-stage reasoning log via Application Insights. This moves AI assistance from a magic box toward an auditable research tool, adhering to the standards of scientific reproducibility and engineering observability simultaneously.

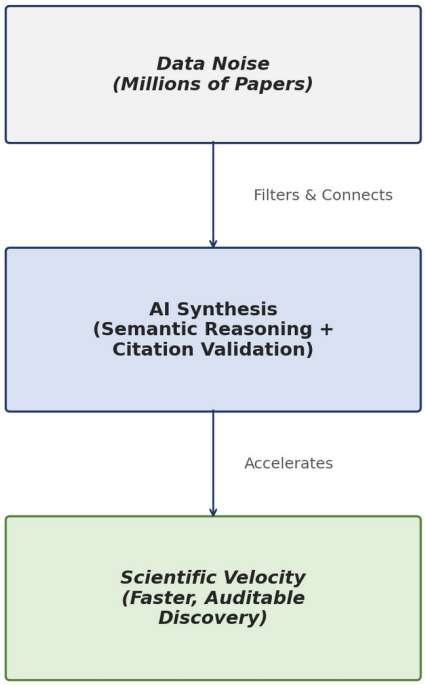

Figure 6. From data noise to scientific velocity: vector retrieval and structured judging convert millions of papers into auditable, citation-grounded synthesis at the pace of research itself.

9. REFERENCES

[1] M. Gridach, J. Nanavati, K. Z. El Abidine, L. Mendes, and C. Mack, "Agentic AI for Scientific Discovery: A Survey of Progress, Challenges, and Future Directions," arXiv preprint arXiv:2503.08979, Mar. 2025.

[2] A. K. Alkan et al., "A Survey on Hypothesis Generation for Scientific Discovery in the Era of Large Language Models," arXiv preprint arXiv:2504.05496, Apr. 2025.

[3] S. Kulkarni and A. Alotaibi, "Scientific Hypothesis Generation and Validation: Methods, Datasets, and Future Directions," arXiv preprint arXiv:2505.04651, May 2025.

[4] Z. Chen et al., "From AI for Science to Agentic Science: A Survey on Autonomous Scientific Discovery," arXiv preprint arXiv:2508.14111, Aug. 2025.

[5] J. Zhang and L. Wei, "Optimizing Retrieval-Augmented Generation (RAG) for Scientific Literature Mining," Knowledge-Based Systems, vol. 289, pp. 112–125, 2025.

[6] D. A. Boiko, R. MacKnight, and G. Gomes, "Emergent autonomous scientific research capabilities of large language models," Nature, vol. 624, pp. 160–168, 2023.

[7] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in Proc. of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23), 2023.

[8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[9] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.

[10] Anthropic, "Claude 4 model family — system documentation," Anthropic technical documentation, 2025–2026.

[11] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, 2023. (Multi-dimensional judging precedent.)

[12] TruEra / TruLens, "TruLens: Evaluation and tracking for LLM applications," open-source documentation, 2023–2025.

[13] Microsoft, ".NET 10 release notes and runtime documentation," Microsoft Learn, 2025–2026.

[14] Qdrant Solutions GmbH, "Qdrant — Open-source vector database, version 1.17," qdrant.tech documentation, 2026.

[15] UglyToad, "PdfPig — read and extract text from PDFs in pure C#," GitHub project documentation, 2025.

[16] Microsoft, "Azure Cosmos DB Vector Search for NoSQL API," Azure documentation, 2025–2026.